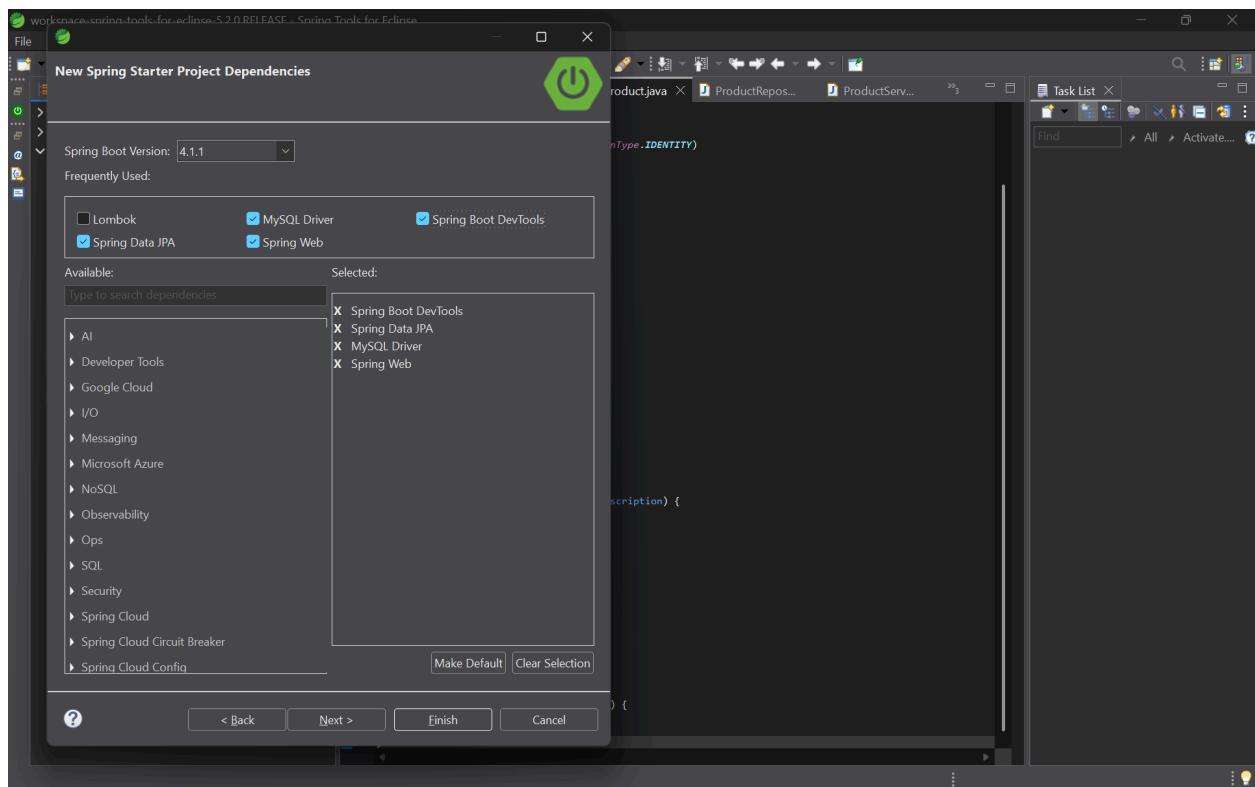
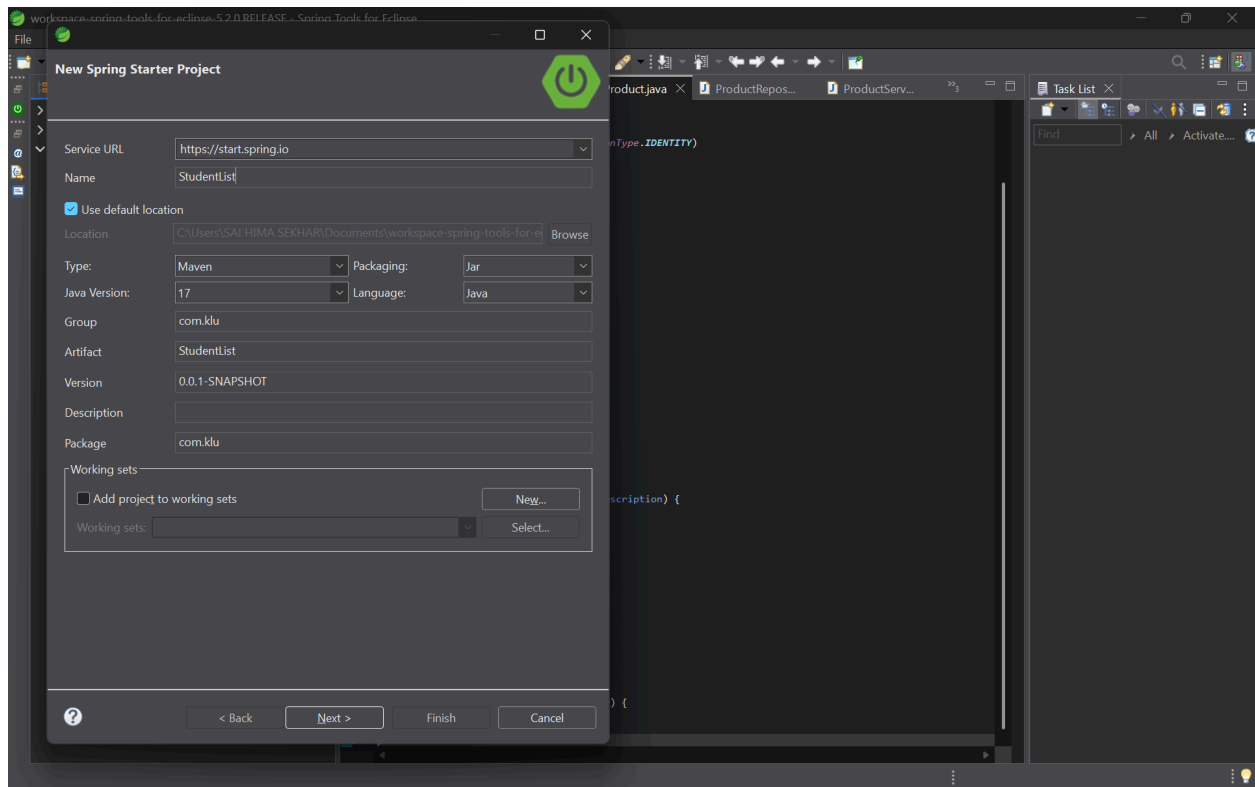
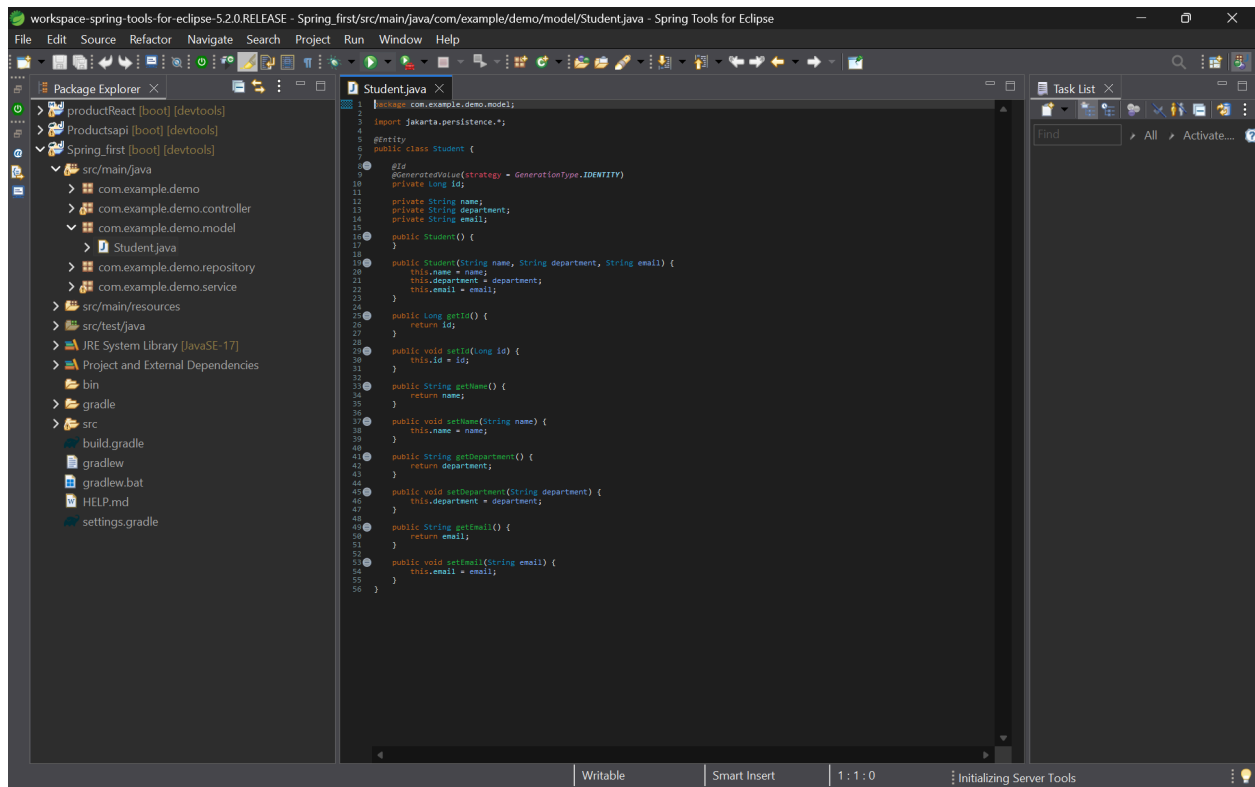


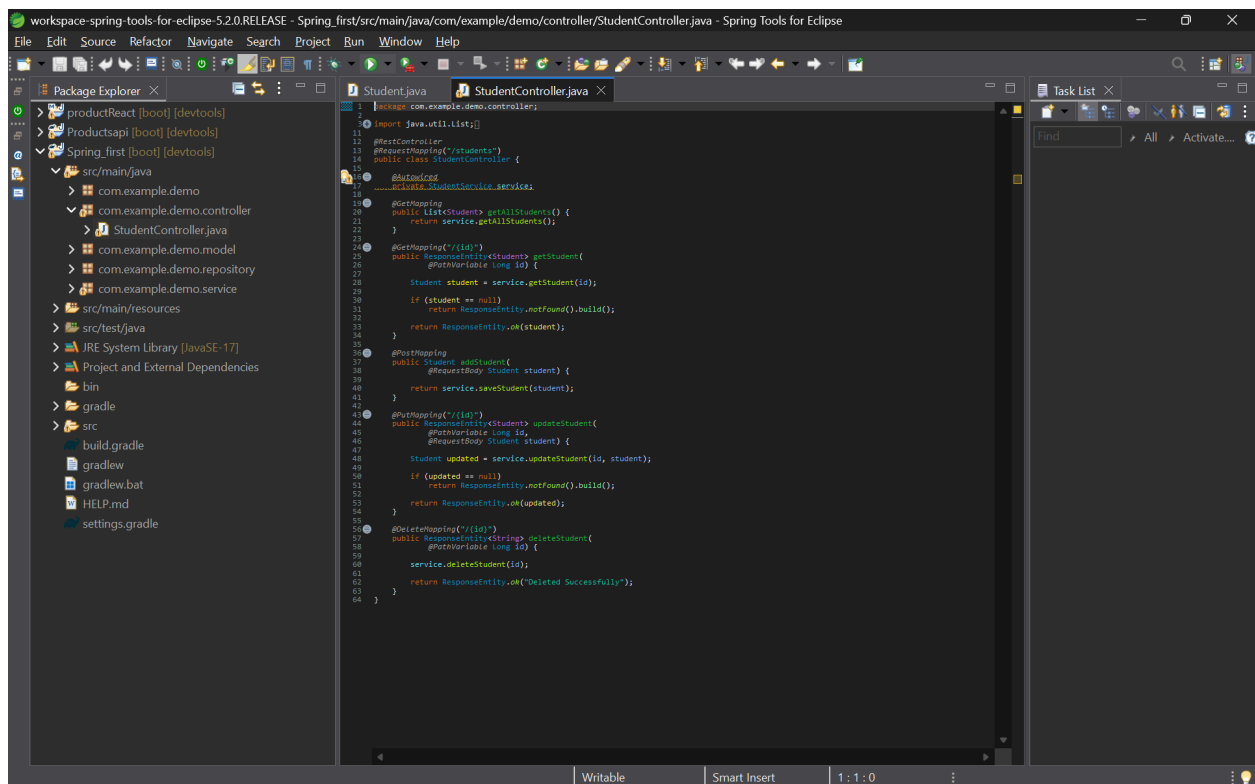
1. CREATING FLORDER



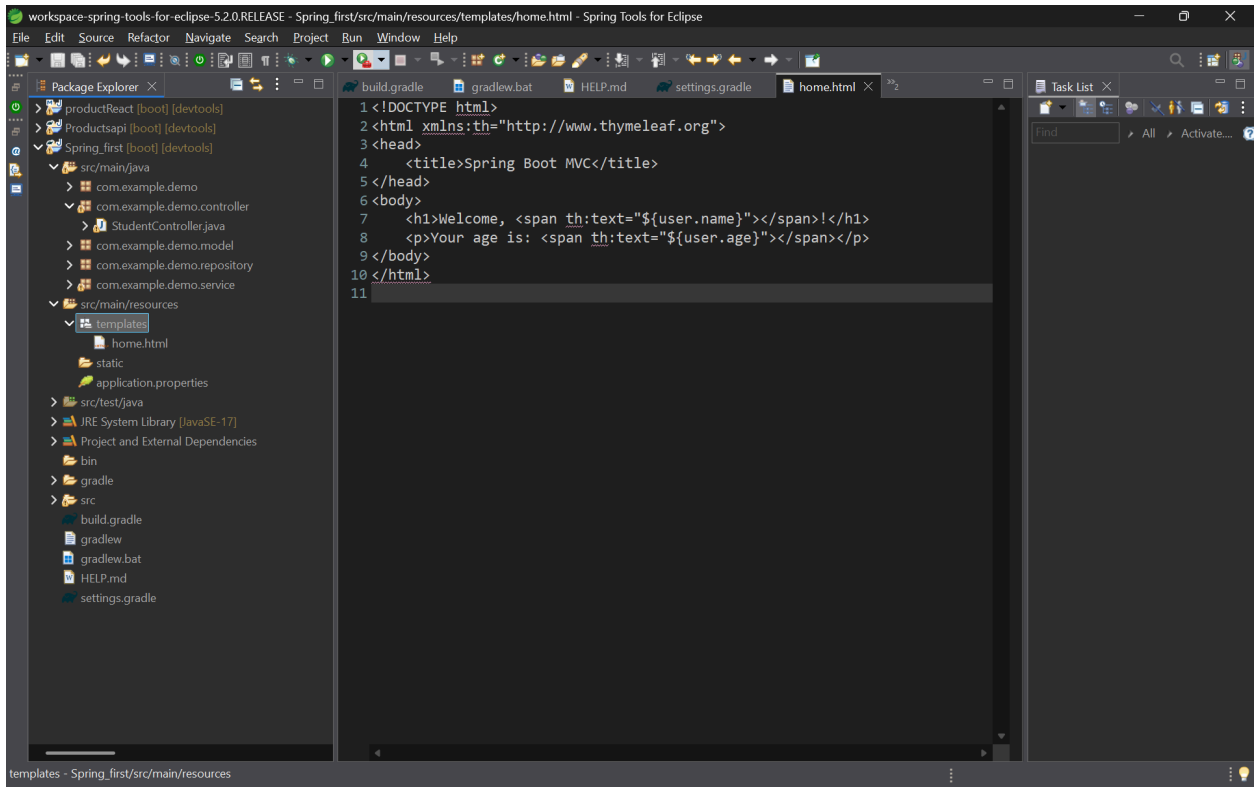
Student.java (Model)



3:StudentController.java



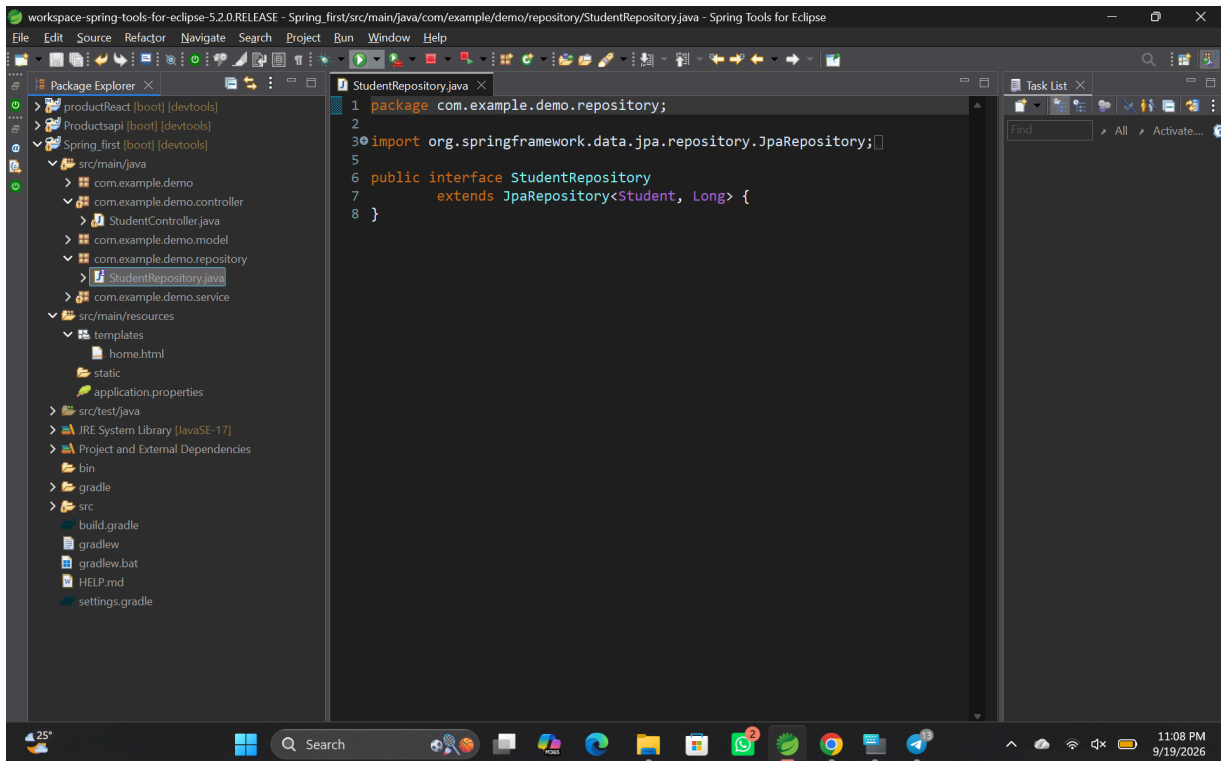
4: Create View (home.html using Thymeleaf)



The screenshot shows the Eclipse IDE with the Package Explorer on the left, the main editor in the center, and the Task List on the right. The Package Explorer shows the project structure with the 'templates' folder selected under 'src/main/resources'. The main editor displays the content of 'home.html', which is an HTML template using Thymeleaf syntax. The Task List is empty.

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <title>Spring Boot MVC</title>
5 </head>
6 <body>
7   <h1>Welcome, <span th:text="${user.name}"></span>!</h1>
8   <p>Your age is: <span th:text="${user.age}"></span></p>
9 </body>
10 </html>
11
```

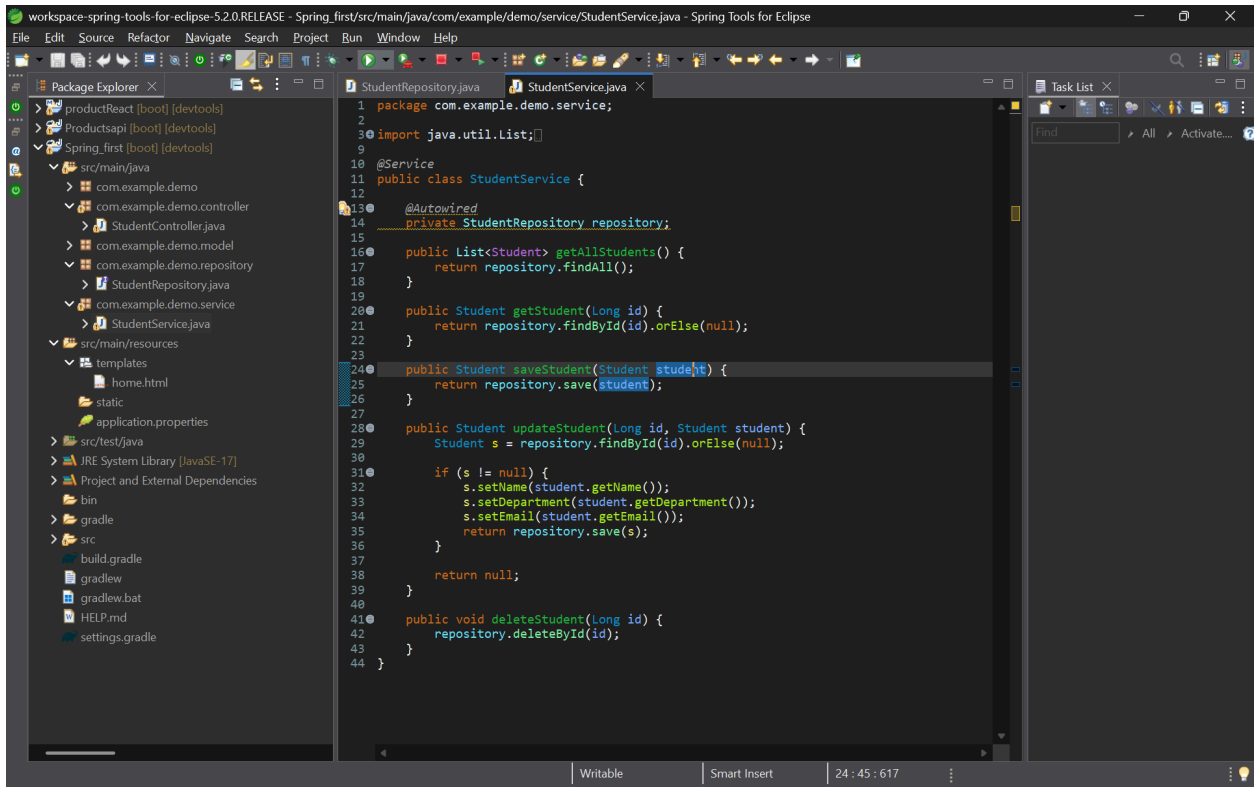
StudentRepository.java



The screenshot shows the Eclipse IDE with the Package Explorer on the left, the main editor in the center, and the Task List on the right. The Package Explorer shows the project structure with the 'StudentRepository.java' file selected under 'src/main/java/com/example/demo/repository'. The main editor displays the content of 'StudentRepository.java', which is a Java interface extending 'JpaRepository'. The Task List is empty.

```
1 package com.example.demo.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6 public interface StudentRepository
7     extends JpaRepository<Student, Long> {
8 }
9
```

StudentService.java



```
1 package com.example.demo.service;
2
3 import java.util.List;
4
5
6
7
8
9
10 @Service
11 public class StudentService {
12
13     @Autowired
14     private StudentRepository repository;
15
16     public List<Student> getAllStudents() {
17         return repository.findAll();
18     }
19
20     public Student getStudent(Long id) {
21         return repository.findById(id).orElse(null);
22     }
23
24     public Student saveStudent(Student student) {
25         return repository.save(student);
26     }
27
28     public Student updateStudent(Long id, Student student) {
29         Student s = repository.findById(id).orElse(null);
30
31         if (s != null) {
32             s.setName(student.getName());
33             s.setDepartment(student.getDepartment());
34             s.setEmail(student.getEmail());
35             return repository.save(s);
36         }
37         return null;
38     }
39
40     public void deleteStudent(Long id) {
41         repository.deleteById(id);
42     }
43
44 }
```